Едвард Студент, Віталій Луцик
Курсова:
Тема курсової: Захищений зв’язок з використанням mTLS

Короткий опис: Реалізувати клієнт-серверну взаємодію на Python з використанням mutual TLS (mTLS) автентифікації, де і клієнт, і сервер автентифікують один одного за допомогою X.509 сертифікатів, виданих власним центром сертифікації. Сервер повинен приймати з’єднання лише від клієнтів з валідним сертифікатом, встановлювати захищений TLS-канал та передавати plain text повідомлення на прикладному рівні. Клієнт повинен ініціювати з’єднання, пройти mTLS-handshake та обмінятися повідомленнями з сервером. У рамках завдання необхідно виконати мережеве перехоплення трафіку за допомогою Wireshark, проаналізувати TLS-handshake (включно з client certificate authentication) та у висновках описати, як такий тип захищеного зв’язку може бути виявлений та контрольований з боку захисних систем.

Практичне розуміння теми/проблеми

а. Де це зустрічається в житті

mTLS суттєво підвищує контроль доступу між клієнтом і сервером, оскільки обидві сторони автентифікують одна одну за X.509 сертифікатами.

Zero Trust Architecture: У сучасних хмарних інфраструктурах (наприклад, Kubernetes service mesh як Istio або Linkerd) mTLS є стандартом для автентифікації мікросервісів.

FinTech та API: Банківські API часто вимагають mTLS для взаємодії з партнерами.

Cybersecurity (C2 канали): Це корисно як для легітимних сервісів, так і для шкідливих каналів керування (Command & Control). Для ШПЗ це привабливо, бо дає сильну взаємну ідентифікацію вузлів, захищений канал “по дефолту” та менше шансів, що випадковий сторонній клієнт зможе під’єднатися до C2.

б. Кому це треба

Розробникам систем з високими вимогами до безпеки: для забезпечення ідентифікації кожного вузла мережі.

SOC/NDR/IDS аналітикам: для розуміння того, як виявляти аномалії в шифрованому трафіку, оскільки mTLS ускладнює мережевий аналіз на прикладному рівні без TLS inspection.

Адміністраторам мереж: для побудови сегментованих зон довіри.

в. Вже існуючі технічні рішення

Варіант A - Python, стандартний ssl: Найкращий варіант для навчальної лабораторії (швидко, мінімум залежностей).

Варіант B - Go: Більш “production-like” реалізація з чистими TLS API та зручною керованістю сертифікатами.

Варіант C - OpenSSL + s_server / s_client: Використовується для первинної валідації сертифікатів та перевірки handshake до написання коду.

Enterprise-рішення: Nginx mTLS configuration, Cloudflare Access, AWS Private CA.

2. Високорівневе рішення

а. Як в ідеалі це мало б працювати

В ідеальній системі реалізація базувалася б на повноцінному Enterprise PKI з автоматизованою ротацією сертифікатів (ACME протокол). Вузол захисту (IDS/IPS) мав би інтегровані можливості TLS Inspection, а

телеметрія збиралася б у реальному часі через Zeek або Suricata, що автоматично витягують метадані сертифікатів (issuer, fingerprint) та корелюють їх з базою відомих загроз через JA3-фінгерпринти.

б. Як ви вирішили це вирішувати в рамках вашого проекту

План і підхід: Реалізація базується на створенні ізольованого середовища з власним Root CA та Python-скриптами для імітації взаємодії "клієнт-сервер". спочатку піднімається PKI інфраструктура через OpenSSL, потім пишеться код на базі бібліотеки ssl, що завершується етапом детекції через аналіз PCAP-файлів. Логічні блоки реалізації:

1. PKI блок: Створення Root CA, випуск серверного та клієнтського сертифікатів з відповідними Extended Key Usage.
2. Кодовий блок: Python-сервер з параметром CERT_REQUIRED та Python-клієнт з валідацією ланцюжка довіри.
3. Аналітичний блок: Перехоплення трафіку в Wireshark та опис ознак (CertificateRequest, Client Certificate).

в. Варіанти захисту (від чого захищає і ні)

Захищає від:

Unauthorized Access: Сервер не пустить клієнта без валідного сертифіката.

Impersonation: Клієнт не буде довіряти недовіреному серверу.

Eavesdropping: Увесь обмін (навіть plain text повідомлення на прикладному рівні) надійно зашифрований у TLS-records.

Не захищає від (залишає артефакти): Навіть коли payload зашифрований, для захисту залишаються артефакти:

сам факт TLS handshake та наявність client certificate authentication;

параметри ClientHello / ServerHello та ланцюжок сертифікатів;

фінгерпринти (JA3/JA3S) та SNI/ALPN;

нетипові порти, частота сесій, обсяг трафіку.

3. Що встигли зробити

а. Чи воно рухається?

На даний момент розроблено загальний план, що робити, та є базова імплементація

Наша базова архітектура

Файли: rootCA.pem/key, server.pem/key, client.pem/key.

Топологія: Сервер (127.0.0.1:8443), Клієнт (окремий процес), Wireshark (capture на loopback).

Маємо код (також він є на гітхабі):

Server.py (сервер):

```python
import socket
import ssl

HOST = "0.0.0.0"
PORT = 8443

# сертифікат і ключ сервера та ca_cert поки лише теоретично
SERVER_CERT = "server.pem"
SERVER_KEY = "server.key"
CA_CERT = "rootCA.pem"

context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile=SERVER_CERT, keyfile=SERVER_KEY)
context.load_verify_locations(cafile=CA_CERT)
context.verify_mode = ssl.CERT_REQUIRED
context.minimum_version = ssl.TLSVersion.TLSv1_2

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as sock:
    sock.bind((HOST, PORT))
    sock.listen(5)
    print(f" Listening on {HOST}:{PORT}")

    with context.wrap_socket(sock, server_side=True) as ssock:
        while True:
            conn, addr = ssock.accept()
            print(f" TLS client connected from {addr}")
            client_cert = conn.getpeercert()
            print(" Client certificate:")
            print(client_cert)
            data = conn.recv(4096)
            print(f"Received: {data.decode(errors='replace')}")
            conn.sendall(b"ACK: secure message received")
            conn.close()
```

Client.py (клієнт):

```python
import socket
import ssl

HOST = "127.0.0.1"
PORT = 8443

# сертифікат і ключ клієнта та ca_cert поки лише теоретично
CLIENT_CERT = "client.pem"
CLIENT_KEY = "client.key"
CA_CERT = "rootCA.pem"

context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH, cafile=CA_CERT)
context.load_cert_chain(certfile=CLIENT_CERT, keyfile=CLIENT_KEY)

context.check_hostname = False
context.minimum_version = ssl.TLSVersion.TLSv1_2

with socket.create_connection((HOST, PORT)) as sock:
    with context.wrap_socket(sock, server_hostname=HOST) as ssock:
        print(" Connected with TLS")
        print("Cipher:", ssock.cipher())
        print("Server cert:", ssock.getpeercert())

        ssock.sendall(b"hello from mtls client")
        data = ssock.recv(4096)
        print(" Response:", data.decode(errors="replace"))
```

Далі після запуску ми плануємо використовувати Wireshark, щоб проаналізувати зв'язок.

б. Як ви можете пояснити питання іншим

Для пояснення механіки mTLS ми плануємо використати аналіз TLS 1.3 Handshake у Wireshark (базуючись на https://wiki.wireshark.org/TLS , https://osqa-ask.wireshark.org/questions/27715/client-authenticated-tls-handshake/ , дебагінг mTLS: https://medium.com/@mintdev/debug-mtls-using-wireshark-b805cf47a1ea, https://www.rfc-editor.org/rfc/rfc8446.html

та відповідних навчальних матеріалах в мережі). Для себе ми виділили наступне:

Ключовим моментом є повідомлення CertificateRequest (тип 13) від сервера. Цю штуку отримуємо завдяки sl.handshake.type == 13 or tls.handshake.type == 13. Очікувана послідовність для аналізу:

Для TLS 1.3, спрощено:

1. ClientHello
2. ServerHello
3. EncryptedExtensions
4. CertificateRequest
5. Certificate (server)
6. CertificateVerify (server)
7. Finished (server)
8. Certificate (client)
9. CertificateVerify (client)
10. Finished (client)
11. Application Data

Саме CertificateRequest є ключовою ознакою того, що сервер вимагає клієнтську автентифікацію. Це прямо випливає з TLS 1.3 handshake

Для виявлення з боку захисту (Detection Levels):

Рівень 1 (Ознаки): TLS-сесія з CertificateRequest, нестандартний Issuer сертифіката.
Рівень 2 (Телеметрія): Логування SNI, ALPN, JA3/JA3S та відбитків сертифікатів через Zeek/Suricata.
Рівень 3 (Аномалії): Один і той самий client cert на багатьох хостах, TLS без SNI, регулярні короткі beacon-like сесії.
Рівень 4 (Контроль): Allowlist корпоративних CA, блокування outbound TLS на нетипових портах, TLS inspection.

Висновок:

На даний момент є розуміння архітектури та інструментарію. Наступні кроки. Обговорення з ментором. Експерименти. Аналіз та генерація фінальних PCAP-файлів, оформлення аналітичної частини звіту. Аналіз помилок та нашого підходу.

Використані джерела:

Cloudflare: What is mutual TLS (mTLS)?
Istio Documentation: Mutual TLS Authentication
RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3
RFC 5280: Internet X.509 PKI Certificate and CRL Profile
MITRE ATT&CK: Technique T1573.002 - Encrypted Channel: Digital Certificates
https://wiki.wireshark.org/TLS - wireshark manual

https://medium.com/@mintdev/debug-mtls-using-wireshark-b805cf47a1ea дебагінг мтлс
https://osqa-ask.wireshark.org/questions/27715/client-authenticated-tls-handshake/
https://www.rfc-editor.org/rfc/rfc8446.html - доки до TLS 1.3